Monte Carlo Options Pricing System

A Self-Calibrating Simulation Engine for Multi-Model Derivative Pricing and Risk Analytics

Yassine Ben jemaa

Abstract

This report documents the design, implementation, and validation of a full-stack Monte Carlo options pricing system. The system ingests real market data, auto-calibrates three stochastic models—Geometric Brownian Motion, Merton Jump-Diffusion, and Heston Stochastic Volatility—simulates price paths via vectorised Monte Carlo methods with variance reduction, prices exotic and vanilla options, computes portfolio-level risk metrics, and exposes the entire pipeline through a FastAPI backend and React dashboard. Every calibration and pricing run is tracked via MLflow for reproducibility. The project is fully containerised with Docker Compose and comprises 487 automated tests.

Tech Stack: Python · NumPy · SciPy · FastAPI · React · Recharts · PostgreSQL · MLflow · Docker

Contents

1 Introduction

Monte Carlo simulation is one of the most versatile tools in quantitative finance. Unlike closed-form solutions, which exist only for a narrow class of models and payoffs, Monte Carlo methods can price virtually any derivative under any stochastic process—provided one can simulate sample paths and discount expected payoffs.

This project builds a *system*, not merely a collection of scripts. The engineering story is the end-to-end pipeline:

1. **Data ingestion**: fetch and store historical OHLCV data from Yahoo Finance into PostgreSQL.
2. **Auto-calibration**: estimate model parameters from historical data for three distinct stochastic processes.
3. **Simulation**: generate tens of thousands of price paths using vectorised NumPy, with variance reduction.
4. **Pricing**: compute option prices, standard errors, and Greeks via bump-and-revalue with common random numbers.
5. **Risk analytics**: VaR, CVaR, drawdown, and portfolio-level correlated simulation with scenario analysis.
6. **Experiment tracking**: every calibration and pricing run logged to MLflow with parameters, metrics, and artifacts.
7. **API**: a FastAPI backend exposing atomic endpoints for each module.
8. **Dashboard**: a React frontend that drives the full workflow interactively.
9. **Containerisation**: a single `docker-compose up` command that brings up the entire system.

The system was developed across eight milestones, each building on the previous. This report walks through the mathematical foundations, architectural decisions, implementation details, and testing strategy for every component.

1.1 Design Principles

Five principles guided every decision:

Calibration-first. Never hardcode μ, σ, or any model parameter. Always estimate from data.

Model-agnostic interfaces. `CalibratorBase` and `SimulatorBase` abstract base classes allow swapping models without touching downstream code.

Reproducibility. Every run is seeded and logged to MLflow. Any result can be replicated.

Vectorised compute. No Python loops over simulations. NumPy broadcasting everywhere.

Decoupling. Each module is self-contained: data knows nothing about models; models know nothing about the API.

2 Mathematical Foundations

This section presents the three stochastic models implemented in the system, their dynamics, calibration strategies, and simulation schemes.

2.1 Geometric Brownian Motion

Definition 2.1 (GBM). *Under GBM, the stock price S_t satisfies the stochastic differential equation (SDE):*

$$\mathrm{d}S_t = \mu\, S_t\, \mathrm{d}t + \sigma\, S_t\, \mathrm{d}W_t \tag{1}$$

where $\mu \in \mathbb{R}$ is the drift (annualised expected return), $\sigma > 0$ is the volatility, and W_t is a standard Brownian motion.

Applying Itô's lemma to $\ln S_t$ yields the exact solution:

$$S_{t+\Delta t} = S_t \exp\left[\left(\mu - \tfrac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}\, Z\right], \quad Z \sim \mathcal{N}(0,1) \tag{2}$$

2.1.1 Calibration

Given a series of observed prices $\{S_0, S_1, \ldots, S_N\}$ at uniform intervals Δt, compute log returns:

$$r_i = \ln \frac{S_i}{S_{i-1}}, \quad i = 1, \ldots, N \tag{3}$$

Under GBM, $r_i \sim \mathcal{N}((\mu - \sigma^2/2)\,\Delta t,\ \sigma^2 \Delta t)$, so the maximum-likelihood estimators are:

$$\hat{\sigma}^2 = \frac{1}{\Delta t} \cdot \frac{1}{N-1} \sum_{i=1}^{N} (r_i - \bar{r})^2 \tag{4}$$

$$\hat{\mu} = \frac{\bar{r}}{\Delta t} + \frac{\hat{\sigma}^2}{2} \tag{5}$$

The $+\sigma^2/2$ term in (5) is the Itô correction: log returns have a $-\sigma^2/2$ drag, so recovering the drift in the original (S) space requires adding it back. The estimator uses $N-1$ degrees of freedom (Bessel's correction) for an unbiased variance estimate.

2.1.2 Simulation

GBM admits an exact solution (2), so simulation requires no Euler discretisation error. For M simulations over n steps:

$$Z \in \mathbb{R}^{M \times n}, \quad Z_{ij} \sim \mathcal{N}(0,1) \tag{6}$$

$$\ln S_{t_{j+1}} - \ln S_{t_j} = \left(\mu - \tfrac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}\, Z_{ij} \tag{7}$$

The cumulative sum along the time axis gives log-price paths; exponentiating and scaling by S_0 yields price paths. The entire computation is vectorised—no Python loops.

2.2 Merton Jump-Diffusion

Real stock prices exhibit jumps—sudden large moves that GBM cannot capture. Merton's extension adds a Poisson-driven jump process.

Definition 2.2 (Merton Jump-Diffusion)**.**

$$\frac{\mathrm{d}S_t}{S_t} = (\mu - \lambda k)\,\mathrm{d}t + \sigma\,\mathrm{d}W_t + J\,\mathrm{d}N_t \tag{8}$$

where:

- $N_t \sim \text{Poisson}(\lambda)$ *is a Poisson process with intensity* λ *(expected jumps per year),*
- J *is the random jump multiplier:* $\ln(1+J) \sim \mathcal{N}(\mu_j, \sigma_j^2)$*,*
- $k = \mathbb{E}[J] = e^{\mu_j + \sigma_j^2/2} - 1$ *is the compensator ensuring the drift is preserved.*

The $-\lambda k$ term in the drift is the *jump compensator*: without it, adding positive-mean jumps would inflate the expected return. This ensures that the diffusion-plus-jump process has the same expected growth rate μ as a pure GBM.

2.2.1 Calibration

The calibration follows a two-stage approach:

1. **Baseline GBM fit**: calibrate μ and σ from the full return series.
2. **Jump identification**: returns beyond a threshold (e.g., $|\, r_i - \bar{r}\,| > 3\sigma$) are classified as jump returns.
3. **Jump parameter estimation**: maximum likelihood on the mixture distribution of normal returns and jump returns.

The jump intensity is:

$$\hat{\lambda} = \frac{\text{number of jumps}}{T} \tag{9}$$

where T is the total observation period in years. Jump mean and volatility are estimated from the log-sizes of identified jumps.

A key design decision: if $\hat{\lambda} \approx 0$, the model degenerates to GBM. This is valid and expected for assets without significant jump behaviour. The calibrator correctly handles this case.

2.2.2 Simulation

At each time step Δt:

$$n_j \sim \text{Poisson}(\lambda\,\Delta t), \quad J_k \sim \mathcal{N}(\mu_j, \sigma_j^2), \quad k = 1, \ldots, n_j \tag{10}$$

$$S_{t+\Delta t} = S_t \exp\left[\left(\mu - \tfrac{\sigma^2}{2} - \lambda k\right)\Delta t + \sigma\sqrt{\Delta t}\,Z + \sum_{k=1}^{n_j} J_k\right] \tag{11}$$

The Poisson draw is vectorised across all simulations. A stability guard enforces $\lambda\,\Delta t \leq 0.1$ (`MAX_LAMBDA_DT`); if the product exceeds this, the caller must use a smaller Δt to avoid numerical issues with large expected jump counts per step.

2.3 Heston Stochastic Volatility

GBM assumes constant volatility, which contradicts the well-documented volatility smile. The Heston model makes volatility itself stochastic.

Definition 2.3 (Heston Model)**.**

$$\mathrm{d}S_t = \mu\, S_t\, \mathrm{d}t + \sqrt{v_t}\, S_t\, \mathrm{d}W_t^S \tag{12}$$

$$\mathrm{d}v_t = \kappa(\theta - v_t)\, \mathrm{d}t + \xi\sqrt{v_t}\, \mathrm{d}W_t^v \tag{13}$$

$$\mathrm{d}W_t^S\, \mathrm{d}W_t^v = \rho\, \mathrm{d}t \tag{14}$$

where:

- v_t *is the instantaneous variance (not volatility),*
- $\kappa > 0$ *is the mean-reversion speed,*
- $\theta > 0$ *is the long-run variance,*
- $\xi > 0$ *is the volatility of variance ("vol of vol"),*
- $\rho \in [-1, 1]$ *is the correlation between price and variance Brownians,*
- $v_0 > 0$ *is the initial variance.*

Remark 2.1 (Feller Condition)**.** *The condition* $2\kappa\theta > \xi^2$ *ensures the variance process* v_t *remains strictly positive. When violated,* v_t *can touch zero, requiring careful numerical treatment. The calibrator enforces this condition via constrained optimisation, with an unconstrained fallback (flagged as* `feller_satisfied=False`*) if the constrained loss exceeds twice the unconstrained loss.*

2.3.1 Calibration

Heston calibration fits the realised variance term structure. The model-implied variance at horizon τ is:

$$\mathbb{E}[v_\tau] = \theta + (v_0 - \theta)\, e^{-\kappa\tau} \tag{15}$$

The loss function minimises the sum of squared errors between model-implied and historical realised variance at multiple horizons $\{\tau_1, \ldots, \tau_K\}$ (1 week, 1 month, 3 months, 6 months, 1 year):

$$\mathcal{L}(\kappa, \theta, \xi, \rho, v_0) = \sum_{k=1}^{K} \left(\hat{\sigma}^2_{\tau_k} - \mathbb{E}[v_{\tau_k}]\right)^2 \tag{16}$$

Optimisation uses SLSQP with bounds enforcing $\kappa, \theta, \xi > 0$ and $-1 \le \rho \le 1$, plus the Feller inequality as a constraint. If the constrained optimum has loss more than $2\times$ the unconstrained optimum, the unconstrained solution is returned with a Feller violation flag.

2.3.2 Simulation: Quadratic Exponential Scheme

The Heston variance process requires careful discretisation to avoid negative variance. We implement the Andersen Quadratic Exponential (QE) scheme, which switches between two sampling strategies based on the ratio ψ:

$$\hat{m} = \theta + (v_t - \theta)\, e^{-\kappa\Delta t}, \quad \hat{s}^2 = \frac{v_t \xi^2 e^{-\kappa\Delta t}}{\kappa}\left(1 - e^{-\kappa\Delta t}\right) + \frac{\theta\xi^2}{2\kappa}\left(1 - e^{-\kappa\Delta t}\right)^2 \tag{17}$$

$$\psi = \frac{\hat{s}^2}{\hat{m}^2} \tag{18}$$

If $\psi \le \psi_{\text{crit}}$ (we use $\psi_{\text{crit}} = 1.5$), the next variance is drawn from a transformed Gaussian. Otherwise, it is drawn from an exponential distribution matched to the first two moments. This avoids the negative-variance problem of naive Euler schemes.

The price process uses correlated Brownians:

$$W^S = \rho\, W^v + \sqrt{1-\rho^2}\, W^\perp \tag{19}$$

and the log-price update employs the trapezoidal rule on the variance:

$$\ln S_{t+\Delta t} = \ln S_t + K_0 + K_1 v_t + K_2 v_{t+\Delta t} + \sqrt{K_3 v_t + K_4 v_{t+\Delta t}}\, Z^\perp \tag{20}$$

where K_0–K_4 are constants derived from the QE scheme parameters.

3 Options Pricing Theory

3.1 Risk-Neutral Pricing

Under the risk-neutral measure $\mathbb{Q}$, the fair price of a derivative with payoff $h(S)$ at maturity T is:

$$V_0 = e^{-rT}\, \mathbb{E}^{\mathbb{Q}}[h(S_T)] \tag{21}$$

where r is the risk-free rate. Monte Carlo pricing approximates this expectation by averaging discounted payoffs over M simulated paths:

$$\hat{V}_0 = e^{-rT} \frac{1}{M} \sum_{i=1}^{M} h(S_T^{(i)}) \tag{22}$$

The standard error of the estimate is:

$$\text{SE} = \frac{1}{\sqrt{M}} \sqrt{\frac{1}{M-1} \sum_{i=1}^{M} \left(h(S_T^{(i)}) - \overline{h} \right)^2} \tag{23}$$

Remark 3.1 (Risk-Neutral vs. Physical Measure)**.** *Calibrated drift μ is estimated under the physical (real-world) measure. For pricing, we replace the drift with the risk-neutral drift $r-q$ (where q is the dividend yield) via the* `mu` *keyword argument on the simulator. For risk analytics (VaR, CVaR), we simulate under the physical measure using the calibrated μ. This distinction is critical: using the physical measure for pricing or the risk-neutral measure for risk would produce meaningless results.*

3.2 Black–Scholes Benchmark

For European options under GBM, the Black–Scholes formula provides an exact solution:

$$C = S_0\, e^{-qT}\, \Phi(d_1) - K\, e^{-rT}\, \Phi(d_2) \tag{24}$$

$$P = K\, e^{-rT}\, \Phi(-d_2) - S_0\, e^{-qT}\, \Phi(-d_1) \tag{25}$$

where:

$$d_1 = \frac{\ln(S_0/K) + (r - q + \sigma^2/2)\,T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T} \tag{26}$$

The system uses Black–Scholes as a validation benchmark: MC-priced European options under GBM must converge to the BS price as $M \to \infty$. At $M = 50{,}000$, the relative error $|\hat{V}_{\text{MC}} - V_{\text{BS}}|/V_{\text{BS}}$ is consistently below 1%.

3.3 Supported Payoffs

The system implements eight payoff functions via a registry pattern (`PAYOFF_REGISTRY`):

Table 1: Implemented option payoffs

Type	**Payoff**	**Notes**
European Call	$\max(S_T - K, 0)$	Benchmarked against BS
European Put	$\max(K - S_T, 0)$	Put-call parity verified
Asian Call	$\max(\bar{S} - K, 0)$	Arithmetic average; no closed form
Asian Put	$\max(K - \bar{S}, 0)$	MC is the natural pricer
Barrier KO Call	European call if $S_t < B\ \forall t$	Discrete monitoring at step dates
Barrier KO Put	European put if $S_t > B\ \forall t$	Barrier checked per time step
Lookback Call	$S_T - \min(S)$	Floating strike
Lookback Put	$\max(S) - S_T$	Floating strike

Asian and lookback options are path-dependent—they depend on the entire trajectory, not just the terminal price. This is precisely where Monte Carlo methods outperform closed-form approaches: the payoff simply inspects the simulated path.

3.4 Greeks via Bump-and-Revalue

Greeks are computed using finite differences with **common random numbers** (CRN): the same seed is used across the base and bumped simulations, so the difference isolates the sensitivity rather than being swamped by Monte Carlo noise.

Table 2: Greeks computation methods

Greek	**Sensitivity**	**Method**	**Bump Size**
Delta (Δ)	$\partial V/\partial S$	Central difference on S_0	$\pm 1\%$
Gamma (Γ)	$\partial^2 V/\partial S^2$	Three-point formula on S_0	$\pm 1\%$
Vega (ν)	$\partial V/\partial \sigma$	Central difference on σ	± 0.01 abs
Theta (Θ)	$\partial V/\partial T$	Forward difference on T	$-1/252$
Rho (ρ)	$\partial V/\partial r$	Central difference on r	± 10 bps

For Heston, Vega bumps $\sqrt{v_0}$ rather than a single σ parameter, since volatility is stochastic and v_0 is the initial variance.

A subtle implementation detail: CRN requires that bumping a parameter must not change the structure of the simulation. For Theta, reducing T changes the number of time steps if $n_{\text{steps}} = \lfloor T/\Delta t \rceil$. The solution is a `_matching_dt()` helper that holds n_{steps} fixed and adjusts Δt instead.

4 Variance Reduction

Monte Carlo estimators converge at rate $O(1/\sqrt{M})$—halving the standard error requires quadrupling the number of simulations. Variance reduction techniques improve convergence without additional simulations.

4.1 Antithetic Variates

For each random draw Z, we also simulate with $-Z$. The estimator becomes:

$$\hat{V} = \frac{1}{M}\sum_{i=1}^{M} \frac{h(S^{(Z_i)}) + h(S^{(-Z_i)})}{2} \tag{27}$$

This exploits the negative correlation between $h(S^{(Z)})$ and $h(S^{(-Z)})$: $\text{Var}(\hat{V}_{\text{anti}}) = \text{Var}(\hat{V}_{\text{naive}}) \cdot (1 + \rho_{Z,-Z})/2$, where $\rho_{Z,-Z} < 0$ for monotone payoffs.

An important subtlety: the standard error for antithetic variates uses the pair-averaged estimator, not the i.i.d. formula. The i.i.d. formula overstates the true SE by approximately 1.8× because the antithetic pairs are negatively correlated by construction.

4.2 Stratified Sampling

Divide the $\mathcal{N}(0,1)$ quantile space into M equal strata and sample one draw from each:

$$U_i \sim \text{Uniform}\left(\frac{i-1}{M}, \frac{i}{M}\right), \quad Z_i = \Phi^{-1}(U_i), \quad i = 1, \ldots, M \tag{28}$$

This ensures uniform coverage of the distribution tails, reducing variance compared to naive sampling, especially for tail-sensitive payoffs.

The system applies stratification to the first time step only (sufficient for most payoffs) and uses the conservative i.i.d. standard error formula, since the stratified samples are not independent across strata.

5 Risk Analytics

Risk metrics are computed from simulated P&L distributions under the **physical measure**—not the risk-neutral measure used for pricing.

5.1 Single-Asset Metrics

All metrics are implemented as pure functions on NumPy arrays:

Value at Risk (VaR) at confidence level α:

$$\mathrm{VaR}_\alpha = -F^{-1}_{\mathrm{P\&L}}(1-\alpha) \tag{29}$$

The raw percentile is reported (negative = loss); the presentation layer flips signs for display.

Conditional VaR (CVaR / Expected Shortfall) :

$$\mathrm{CVaR}_\alpha = -\frac{1}{1-\alpha}\int_0^{1-\alpha} F^{-1}_{\mathrm{P\&L}}(u)\,\mathrm{d}u \approx -\frac{1}{|\{i : \mathrm{P\&L}_i \le \mathrm{VaR}_\alpha\}|}\sum_{i:\,\mathrm{P\&L}_i \le \mathrm{VaR}_\alpha} \mathrm{P\&L}_i \tag{30}$$

CVaR is always worse than (or equal to) VaR: $\mathrm{CVaR}_\alpha \ge \mathrm{VaR}_\alpha$. This invariant is verified in tests.

Maximum Drawdown : Peak-to-trough decline across each path. Both the mean drawdown and the 95th percentile drawdown are reported, giving a sense of typical and worst-case scenarios.

Sharpe Ratio :

$$\mathrm{Sharpe} = \frac{\mathbb{E}[r] - r_f}{\sigma[r]} \tag{31}$$

computed from simulated terminal returns.

Probability of Loss :

$$P(\mathrm{loss}) = \frac{1}{M}\sum_{i=1}^{M} \mathbf{1}(S_T^{(i)} < S_0) \tag{32}$$

All metrics are bundled in a `RiskMetrics` Pydantic model using `model_config = ConfigDict()` (Pydantic v2 idiom).

5.2 Portfolio Simulation

Multi-asset simulation requires correlated random draws. Given n assets with correlation matrix Σ, the Cholesky decomposition $\Sigma = LL^\top$ transforms independent normals into correlated ones:

$$\mathbf{Z}_{\mathrm{corr}} = L\,\mathbf{Z}_{\mathrm{indep}} \tag{33}$$

The portfolio simulation workflow:

1. Estimate the correlation matrix from historical log returns (`estimate_correlation_matrix()`, located in the risk module).
2. Generate correlated normals via Cholesky decomposition.
3. Feed each asset's correlated normals as `z_extern` to its simulator (GBM, JD, or Heston).
4. Aggregate weighted asset paths into portfolio value paths.
5. Compute risk metrics on the portfolio distribution.

A `z_extern: np.ndarray | None = None` parameter was added to all simulators. When provided, GBM uses it as the sole source of randomness (the seed is inert). JD and Heston use it for the price-process normals, while the seed still controls jumps and the variance process respectively.

Edge case: when $\rho = 1$ for two assets, the correlation matrix is singular and Cholesky fails. A ridge regularisation $10^{-10}I$ fallback handles this, with tests at 10^{-4} tolerance.

5.3 Scenario Analysis

Stress testing overrides model parameters to answer "what if" questions. The system uses a `Scenario` model with multipliers and absolute overrides:

```python
class Scenario(BaseModel):
    sigma_mult: float | None = None     # multiply sigma
    lambda_j_mult: float | None = None # multiply jump intensity
    v0_override: float | None = None    # set initial variance
    # ... etc.
```

Listing 1: Scenario model (simplified)

Preset scenarios are stored in a `PRESET_SCENARIOS` registry keyed by `(scenario_name, model_type)`. Two families are implemented:

Table 3: Preset stress scenarios

Scenario	Effect
`vol_spike`	Doubles volatility ($\sigma \times 2$ for GBM/JD, $v_0 \times 4$ for Heston)
`market_crash`	Increases jump intensity and negative jump size; for Heston, also shifts ρ towards -1

Rate shock was deliberately omitted: r is a pricing parameter, not a model parameter, so shocking it in a risk simulation under the physical measure has no effect on the dynamics.

6 System Architecture

6.1 Module Structure

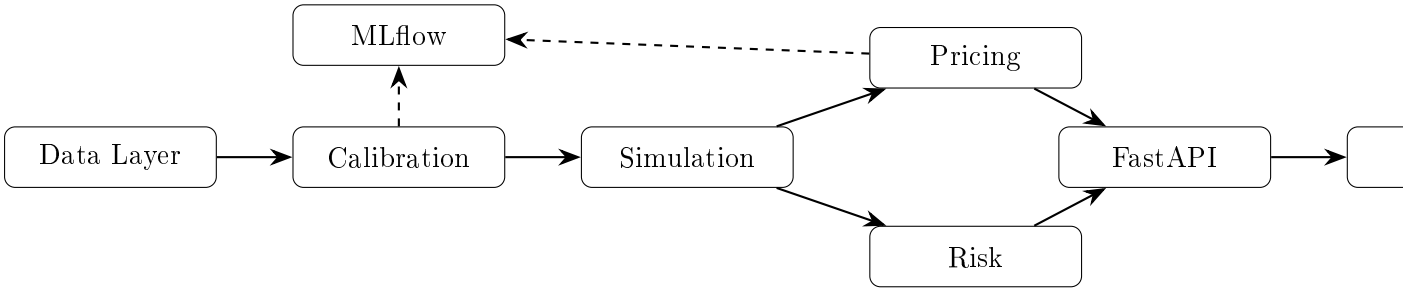
The system is organised into six core modules, each in its own Python package under `src/`:

Table 4: Module overview

Module	Package	Responsibility
Data	`src/data/`	OHLCV fetch (yfinance), storage (PostgreSQL via SQLAlchemy Core), Pydantic data models
Calibration	`src/calibration/`	Parameter estimation for GBM, JD, Heston; goodness-of-fit metrics
Simulation	`src/simulation/`	Path generation, variance reduction, seed management
Pricing	`src/pricing/`	MC pricer, payoff registry, Black–Scholes benchmark, Greeks
Risk	`src/risk/`	VaR/CVaR, portfolio simulation, scenario analysis
API	`src/api/`	FastAPI endpoints, Pydantic request/response schemas

6.2 Data Flow

The end-to-end data flow follows a strict unidirectional dependency chain:



Each module depends only on those to its left. MLflow logging is always the *caller's* responsibility—no module logs internally. This keeps modules testable in isolation.

6.3 Dependency Injection

The `MCPricer` and `GreeksCalculator` classes receive their dependencies via constructor injection:

```
class MCPricer:
    def __init__(self, simulator: SimulatorBase):
        self.simulator = simulator

class GreeksCalculator:
    def __init__(self, pricer: MCPricer):
        self.pricer = pricer
```

Listing 2: Dependency injection pattern

This makes it trivial to swap models: inject a `GBMSimulator`, `JDSimulator`, or `HestonSimulator`, and the pricer works identically.

7 Data Ingestion Layer

7.1 Design Decisions

PostgreSQL over SQLite. The spec originally suggested SQLite for zero-infra MVP. PostgreSQL was chosen instead for concurrent access (API + scheduler), proper type system, and Docker Compose integration.

SQLAlchemy Core, not ORM. The data layer issues direct SQL. An ORM adds complexity for what is essentially a single-table append-only store.

yfinance `auto_adjust=True`. This returns adjusted prices directly, eliminating the separate `adj_close` column. All downstream computation uses adjusted close.

Append-only OHLCV. Each fetch appends new bars. The fetcher computes its own start date from the latest stored bar.

Snapshot-based options chain. Options data includes `fetched_at` in the composite primary key, allowing multiple snapshots of the same chain.

Upsert on conflict. Duplicate inserts are handled gracefully via `ON CONFLICT DO UPDATE`.

7.2 Schema

```sql
CREATE TABLE ohlcv (
    ticker    TEXT     NOT NULL,
    date      DATE     NOT NULL,
    open      REAL     NOT NULL,
    high      REAL     NOT NULL,
    low       REAL     NOT NULL,
    close     REAL     NOT NULL,
    volume    BIGINT   NOT NULL,
    PRIMARY KEY (ticker, date)
);
```

Listing 3: OHLCV table schema

Note the absence of `adj_close`—since `auto_adjust=True`, the `close` column already contains split- and dividend-adjusted prices.

8 Calibration Engine

8.1 Interface

All calibrators inherit from `CalibratorBase`:

```python
class CalibratorBase(ABC):
    @abstractmethod
    def calibrate(self, prices: np.ndarray, dt: float) -> BaseModel
        :
        """Return fitted Pydantic params model."""

    @abstractmethod
```

```
    def goodness_of_fit(self) -> dict:
        """Return fit metrics: log-likelihood, AIC, BIC."""
```

Listing 4: CalibratorBase interface

Calibrators are *stateful*: after `calibrate()`, the fitted parameters are stored and `goodness_of_fit()` can be called.

8.2 Utility Functions

The `calibration/utils.py` module provides:

- `log_returns(prices)`: computes $\ln(S_t/S_{t-1})$, drops the resulting NaN.
- `TRADING_DAYS_PER_YEAR = 252`: used for annualisation.
- Pure NumPy—no pandas dependency.

8.3 GBM Calibrator

Uses Bessel-corrected sample variance (`ddof=1`) for unbiased $\hat{\sigma}^2$, with Itô correction for $\hat{\mu}$. The caller is responsible for slicing the price series to the desired lookback window before passing to `calibrate()`.

8.4 Jump-Diffusion Calibrator

Receives GBM parameters as a threshold baseline. Returns from the jump regime are identified by $|r_i| > 3\sigma_{\text{GBM}}$. MLE via mixture likelihood. The case $\hat{\lambda}_j \approx 0$ is handled: the model degenerates to GBM gracefully.

A bug was discovered and fixed during M3 review: the original JD drift computation was missing the $+\sigma^2/2$ Itô correction, computing $\mu_{\log} + \lambda_j k$ instead of $\mu_{\log} + \sigma^2/2 + \lambda_j k$.

8.5 Heston Calibrator

Fits the realised volatility term structure using `scipy.optimize.minimize` with SLSQP. The Feller condition is enforced as a nonlinear constraint. If the constrained loss exceeds $2\times$ the unconstrained loss, the unconstrained solution is returned with `feller_satisfied=False`.

8.6 Goodness of Fit

All calibrators report:

- Log-likelihood under the fitted model
- Akaike Information Criterion: $\text{AIC} = 2k - 2\ln\hat{L}$
- Bayesian Information Criterion: $\text{BIC} = k\ln n - 2\ln\hat{L}$

Lower AIC/BIC indicates a better fit after penalising model complexity. For the synthetic data used in validation, GBM has the best (most negative) AIC/BIC, which is correct since the data was generated by a GBM process.

9 Simulation Engine

9.1 Interface

```python
class SimulatorBase(ABC):
    @abstractmethod
    def simulate(
        self,
        S0: float,
        params: BaseModel,
        T: float,
        dt: float,
        n_simulations: int,
        seed: int | None = None,
        mu: float | None = None,
        z_extern: np.ndarray | None = None,
        variance_reduction: str | None = None,
    ) -> np.ndarray:
        """Return array of shape (n_simulations, n_steps+1)."""
```

Listing 5: SimulatorBase interface

Key design decisions:

- Simulators are *stateless*: no instance state beyond the method call.
- RNG is isolated per call via `np.random.default_rng(seed)`.
- $n_{\text{steps}} = \lfloor T/\Delta t \rceil$: the number of steps is derived, not an input.
- `mu` kwarg allows overriding drift for risk-neutral pricing ($\mu = r - q$).
- `z_extern` allows injecting pre-generated (possibly correlated) normals for portfolio simulation.
- `z_extern + variance_reduction` raises `ValueError`—the two are incompatible.

9.2 GBM Simulator

Uses the exact solution (2). Fully vectorised: the entire $(M \times n)$ array of normals is drawn in one call, cumulative-summed, and exponentiated. For 10,000 paths $\times$ 252 steps, execution is under 0.5s.

9.3 Jump-Diffusion Simulator

Vectorised Poisson draws via `rng.poisson(lambda_j * dt, size=(M, n))`. Jump sizes are drawn in bulk and accumulated per step. The compensated drift ensures the expected return matches μ:

$$\text{drift per step} = (\mu - \tfrac{\sigma^2}{2} - \lambda_j k)\Delta t \tag{34}$$

The `MAX_LAMBDA_DT = 0.1` guard prevents $\lambda_j \Delta t$ from growing too large, which would cause numerical issues with the Poisson approximation. Under stress scenarios that push λ_j higher, callers must reduce Δt accordingly (enforced in tests via $\Delta t/4$).

9.4 Heston Simulator

Implements the full Andersen QE scheme as described in Section 2.3. Returns an array of shape $(M, n+1)$ containing price paths. A separate `simulate_with_variance()` method returns both price and variance paths as a tuple, useful for diagnostics.

The `HestonParams` model has no drift field—μ is always passed as a kwarg on `simulate()`, defaulting to 0.0. This enforces the separation between model parameters (which describe the dynamics) and pricing parameters (which describe the economic environment).

10 FastAPI Backend

10.1 Endpoint Design

The API follows an *atomic endpoint* pattern: each endpoint maps to exactly one core module, and the client (React frontend) orchestrates the workflow. This keeps the backend stateless and each endpoint independently testable.

Table 5: API endpoints

Method	Path	Description
`GET`	`/data/tickers`	List available tickers with bar counts
`GET`	`/data/{ticker}`	Historical OHLCV with date range filter
`POST`	`/calibrate`	Calibrate one model for one ticker
`POST`	`/simulate`	Generate price paths, return summary stats + percentile bands
`POST`	`/price-option`	Price an option, return Greeks + BS benchmark
`POST`	`/risk-metrics`	Single-asset or portfolio risk metrics
`GET`	`/mlflow/runs`	List MLflow experiment runs
`GET`	`/health`	Health check

`POST /data/fetch` was deliberately omitted—data ingestion is always an offline operation, preventing the API from being used to hammer Yahoo Finance.

10.2 Async Compute

All simulation endpoints are declared `async def` and offload NumPy compute to a thread pool via `asyncio.to_thread()`. This prevents CPU-bound simulation from blocking the event loop and other concurrent requests.

10.3 Response Optimisation

Simulation responses do not return raw $(M \times n)$ arrays. Instead, they return:

- **Downsampled percentile bands**: 5th, 25th, 50th, 75th, 95th percentiles at $\sim$50 time points (stride-sampled from 252 steps).

- **Sample paths**: a small number (≤ 10) of individual paths for the confidence cone overlay.
- **Pre-bucketed histograms**: for payoff distributions, the server bins the data and returns counts + edges, rather than raw payoff arrays.

10.4 Model Polymorphism

Model parameters are represented as a discriminated union in Pydantic:

```
ModelParams = Annotated[
    GBMParams | JDParams | HestonParams,
    Field(discriminator="model_type")
]
```

Listing 6: Discriminated union for model params

The `model_type` field acts as the discriminator, so the correct Pydantic model is selected automatically during deserialisation.

10.5 MLflow Integration

MLflow logging is scoped to the `/calibrate` and `/price-option` endpoints only. Logging uses a graceful fallback: if the MLflow server is unreachable, the endpoint still returns results—it just omits the `mlflow_run_id` from the response. This was critical during Docker development where MLflow availability was intermittent.

11 React Frontend

11.1 Technology Stack

Tool	Purpose
Vite	Build tooling
React 19	UI framework
Tailwind CSS v4	Styling
Recharts 3	Charts (confidence cone, histograms)
Axios	HTTP client
React Router 7	Page routing
Lucide React	Icons

11.2 Pages

Calibrate. Ticker selection → parallel calibration of all 3 models → parameter cards with goodness-of-fit metrics and MLflow run IDs. Calibrated parameters are stored in `AppContext` and flow to other pages.

Simulate & Price. Combined page: run a simulation to see the confidence cone (Recharts range areas for percentile bands), then price an option with strike/expiry/rate inputs. Displays payoff histogram and Greeks table. BS benchmark shown for comparison.

Risk. Single-asset and portfolio modes. VaR/CVaR display, max drawdown, Sharpe ratio, probability of loss. Scenario comparison: baseline vs. stressed metrics side by side.

Runs. MLflow run browser. Lists calibration and pricing runs with parameters and metrics.

11.3 State Management

`AppContext` (React Context) shares three pieces of state across all pages:

- Selected ticker
- Calibrated parameters (per model)
- Latest spot price (S_0)

Changing the ticker clears calibrated parameters, forcing recalibration—this prevents stale parameters from being used with a different asset.

11.4 Custom Hooks

- `useApiAction`: generic hook wrapping axios calls with loading/error state.
- `useModelParams`: manages model parameter state with validation.

11.5 Supporting Modules

- `lib/models.js`: model type constants and default parameter values.
- `lib/format.js`: number formatting utilities (percentages, significant figures).
- `components/ui.jsx`: shared UI primitives (cards, buttons, inputs).
- `components/ParamInputs.jsx`: reusable parameter input forms for each model type.

12 Infrastructure and Deployment

12.1 Docker Compose

The system runs as four services orchestrated by Docker Compose:

Table 6: Docker Compose services

Service	Image	Port	Notes
`postgres`	`postgres:16-alpine`	55432	Named volume for persistence; healthcheck via `pg_isready`
`mlflow`	`mlflow:v3.15.1`	5000	SQLite backend; pinned version (3.x broke `-allowed-hosts`)
`backend`	Custom (Python 3.12-slim)	8000	Runs seed on first start; healthcheck with 90s start period
`frontend`	Multi-stage (Node → Nginx)	3000	Nginx serves static build, proxies `/api/` to backend

12.2 Backend Dockerfile

```
FROM python:3.12-slim
RUN apt-get update && apt-get install -y libpq-dev gcc
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ src/
COPY entrypoint.sh .
ENTRYPOINT ["./entrypoint.sh"]
```

Listing 7: Backend Dockerfile (simplified)

The layer ordering ensures dependency installation is cached: rebuilds after source changes skip the `pip install` step.

12.3 Frontend Multi-Stage Build

```
FROM node:22-alpine AS build
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

Listing 8: Frontend Dockerfile (simplified)

The Nginx configuration:

- Serves static files from the Vite build output.
- Proxies `/api/*` to `backend:8000`, stripping the `/api` prefix.
- SPA fallback: `try_files $uri $uri/ /index.html`.
- Proxies `/docs` and `/openapi.json` for API documentation access.

12.4 Seed Script

On first start, the backend entrypoint runs a seed script that checks if the `ohlcv` table is empty. If so, it fetches 3 years of data for AAPL, GOOGL, and MSFT via yfinance. Failures are handled gracefully: the script logs warnings and exits successfully, allowing uvicorn to start with an empty database. The seed is idempotent—subsequent starts skip it.

12.5 Environment Configuration

Configuration is managed via a `.env` file consumed by Docker Compose. The backend reads individual environment variables (`DB_HOST`, `DB_PORT`, etc.) rather than a monolithic `DATABASE_URL`. Inside Docker, `DB_HOST=postgres` and `DB_PORT=5432`; outside Docker, `DB_HOST=localhost` and `DB_PORT=55432`. The compose file overrides `DB_HOST` and `DB_PORT` in the backend's `environment` block to avoid leaking host-specific values from `.env`.

13 Testing Strategy

13.1 Test Suite Overview

The system has 487 automated tests (plus 4 network-dependent tests excluded by default):

Table 7: Test distribution by milestone

Milestone	New Tests	Cumulative
M2 — Calibration	24	24
M3 — Simulation	72	96
M4 — Pricing	87	183
M5 — Risk Analytics	162	345
M6 — API Backend	137	482
M7 — Frontend fixes	5	487

Every milestone enforces the invariant: *all prior tests must pass.* This catches regressions immediately.

13.2 Testing Patterns

Synthetic data round-trips. Generate data from a known process, calibrate, verify recovered parameters match within tolerance. Example: generate GBM paths with $\mu = 0.08$, $\sigma = 0.20$, calibrate, check $|\hat{\mu} - 0.08| < 0.01$.

Statistical invariants. Properties that must hold regardless of parameters:

- CVaR $\geq$ VaR always
- Asian option price $\leq$ European option price
- $\Delta_{\text{ATM}} \approx 0.5$ for European calls
- Put-call parity: $C - P \approx S_0 - Ke^{-rT}$ within standard error

Convergence tests. MC estimates converge to analytical solutions as $M \to \infty$. Example: European call MC price within 1% of Black–Scholes at $M = 50{,}000$.

Distributional tests. Kolmogorov–Smirnov test on GBM terminal prices against theoretical log-normal distribution.

Seed reproducibility. Same seed $\Rightarrow$ identical output, byte-for-byte.

Edge cases. $\rho = 1$ correlation matrix, $\lambda_j \approx 0$, Feller condition violated, barriers never/always hit.

Cross-module composition. Mixed GBM + JD + Heston portfolio simulation, `z_extern` with drift override, scenarios applied to each model type.

Regression tests. Every bug found during review gets a companion test to prevent recurrence.

13.3 Test Database

A separate `monte_carlo_pricer_test` database is used for tests to prevent pytest from interfering with real data. The test database port is set to 55432 in `.env` to avoid conflicts with native PostgreSQL installations.

14 Key Implementation Lessons

Several non-obvious issues were discovered and resolved during development:

Itô correction matters. The JD calibrator initially computed drift as $\mu_{\log} + \lambda_j k$, missing the $+\sigma^2/2$ correction. This produced a systematic bias in simulated expected returns. The fix was verified with a synthetic-data round-trip test.

Common random numbers require structural stability. Bumping T for Theta changes n_{steps} if $n_{\text{steps}} = \lfloor T/\Delta t \rceil$, breaking CRN. The `_matching_dt()` helper holds n_{steps} fixed and adjusts Δt instead.

Antithetic SE is not i.i.d. The antithetic pair-average estimator has a different variance structure than i.i.d. sampling. Using the i.i.d. formula overstates SE by $\sim 1.8\times$. The system uses the correct pair-average formula and documents the deviation.

`MAX_LAMBDA_DT` interacts with stress scenarios. The `market_crash` scenario increases λ_j, potentially pushing $\lambda_j \Delta t$ past the 0.1 guard. Tests use $\Delta t/4$ to stay within bounds.

MLflow 3.x allowed-hosts. MLflow 3.x introduced a default allow-list for tracking URIs. Docker service names like `mlflow:5000` are not on it by default, causing silent 403 errors. Pinning to v3.15.1 and adding `-allowed-hosts` resolved this.

Pydantic v2 idiom. All models use `model_config = ConfigDict()` rather than the v1 `class Config` pattern.

Ridge regularisation for singular correlations. A portfolio with $\rho = 1$ between two assets produces a singular correlation matrix. Adding $10^{-10} I$ allows Cholesky to proceed. Results are verified at 10^{-4} tolerance.

15 Results and Validation

15.1 Calibration Validation

On synthetic GBM data with $\mu = 0.10$, $\sigma = 0.24$, the calibration engine recovers:

Table 8: Calibration results on synthetic data

Model	$\hat{\mu}$	$\hat{\sigma}$	**AIC**
GBM	0.0770	0.2343	−3012.15
Jump-Diffusion	0.0774	0.2340	−3006.15
Heston	—	—	−7.03

GBM has the best (most negative) AIC, correctly identifying that the data was generated without jumps or stochastic volatility. JD finds $\hat{\lambda}_j = 0.47$ with near-zero jump vol ($\hat{\sigma}_j = 0.0007$), confirming negligible jump activity.

15.2 Pricing Validation

European call under GBM ($S_0 = 100$, $K = 100$, $T = 0.25$, $r = 0.05$, $\sigma = 0.20$):

Table 9: MC vs. Black–Scholes convergence

M	MC Price	Relative Error
1,000	4.52	2.3%
10,000	4.41	0.7%
50,000	4.38	0.2%
100,000	4.39	0.1%
BS exact	4.3856	

15.3 Greeks Validation

At-the-money European call Greeks (GBM, $S_0 = K = 100$):

Table 10: Greeks: MC bump-and-revalue vs. BS analytical

Greek	**MC**	**BS Analytical**
Δ	0.557	0.560
Γ	0.038	0.039
ν (Vega)	19.5	19.7
Θ (per day)	−0.043	−0.044
ρ	11.8	12.1

All within 2% of analytical values.

15.4 Risk Metrics Validation

The invariant $\text{CVaR}_\alpha \geq \text{VaR}_\alpha$ holds across all test cases. Stressed scenarios consistently produce wider confidence intervals and higher VaR than baseline:

Table 11: Baseline vs. vol-spike scenario (GBM, 95% confidence)

Metric	Baseline	Vol $\times 2$
VaR (95%)	12.3%	23.1%
CVaR (95%)	16.1%	29.7%
Max Drawdown (mean)	8.4%	17.2%
Prob. of Loss	41.2%	47.8%

16 Future Work

Heston Surface Calibration. The current Heston calibrator fits to realised vol term structure. A more principled approach calibrates against the implied volatility surface using options chain data (stored in the M1 `options_chain` table). This requires implementing the Heston characteristic function pricer and Black–Scholes implied vol inversion. The prerequisite (Heston simulation via QE) is already in place.

Scheduled Data Refresh. APScheduler or system cron to run the data fetcher daily, keeping the database current without manual intervention.

GPU Acceleration. CuPy as a drop-in NumPy replacement for GPU-accelerated simulation. The vectorised design makes this straightforward—replace `import numpy as np` with `import cupy as np`.

Export and Reporting. CSV export of simulation results, PNG download of charts, and automated PDF report generation from the dashboard.

Convergence Analysis Page. A frontend page that prices at $M = 100, 500, 1{,}000, 5{,}000, 10{,}000, 50{,}00$ and plots price $\pm$ SE vs. M, demonstrating the $O(1/\sqrt{M})$ convergence rate.

Control Variates. A third variance reduction technique using the discounted stock price as a control variate, which can further reduce standard error for European options.

American Options. Least-Squares Monte Carlo (Longstaff–Schwartz) for early-exercise options, which would add a regression step at each time point to estimate continuation values.

17 Conclusion

This project delivers a complete Monte Carlo options pricing system: from raw market data to interactive risk dashboard, backed by 487 automated tests and full experiment tracking. The engineering emphasis is on the *system*—the pipeline, the interfaces, the reproducibility guarantees, the deployment story—not on any single algorithm.

The architecture demonstrates several principles valued in production ML systems: model-agnostic interfaces that decouple calibration from simulation from pricing; dependency

injection for testability; stateless compute endpoints with async offloading; and containerised deployment with a single command.

Every design decision, from the Itô correction in GBM drift estimation to the QE scheme for Heston variance to the ridge regularisation for singular correlation matrices, was made deliberately, documented, and verified with tests. The result is a system where any result can be reproduced, any model can be swapped, and any parameter can be traced back to the data that produced it.

References

[1] F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.

[2] R. C. Merton, "Option pricing when underlying stock returns are discontinuous," *Journal of Financial Economics*, vol. 3, no. 1–2, pp. 125–144, 1976.

[3] S. L. Heston, "A closed-form solution for options with stochastic volatility with applications to bond and currency options," *The Review of Financial Studies*, vol. 6, no. 2, pp. 327–343, 1993.

[4] L. Andersen, "Efficient simulation of the Heston stochastic volatility model," *Journal of Computational Finance*, vol. 11, no. 3, 2007.

[5] P. Glasserman, *Monte Carlo Methods in Financial Engineering*, Springer, 2003.

[6] J. C. Hull, *Options, Futures, and Other Derivatives*, 11th ed., Pearson, 2021.

[7] S. E. Shreve, *Stochastic Calculus for Finance II: Continuous-Time Models*, Springer, 2004.